```
$a = 'a val';
$aRef = &$a;

unset($aRef);
var_dump($aRef);
var_dump($a);

Output:
Warning: Undefined variable $aRef
NULL
string(5) "a val"
```

However, unsetting a referred variable has a slightly less intuitive result:

```
$a = 'a val';
$aRef = &$a;

unset($a);
var_dump($aRef);
var_dump($a);

Output:
string(5) "a val"
Warning: Undefined variable $a
NULL
```

As you can see, even though `$a` no longer exists, its value (the underlying *zval*) still has a reference to it as far as PHP is concerned, so the data remains available to `$aRef`.

Variable Variables

Earlier in the chapter, we looked at the rules for variables. Remember that a variable is composed of a $ symbol and a name formed from (legal) characters? Well, here's a weird trick: the name can be a variable too! This form of variable is called (surprisingly) a *variable variable*, and they look like this:

```
$val = 'some value';
$name = 'val';
var_dump($$name);
```